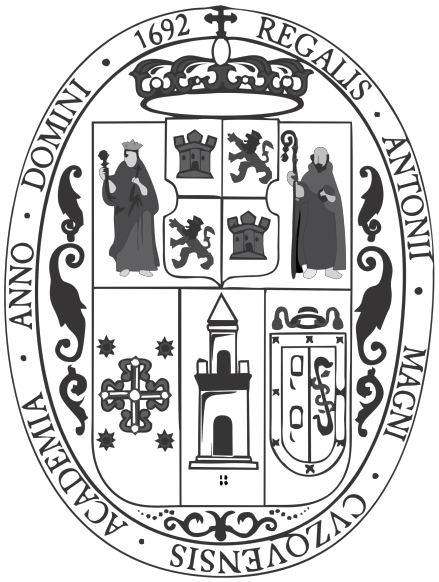
UNIVERSIDAD NACIONAL SAN ANTONIO ABAD DEL CUSCO

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

Escuela Profesional de Ingeniería Informática y de Sistemas





Informe proyecto

Aplicativo de rastreo espacial

Curso: Sistemas Embebidos

Docente: Jose Mauro Pillco Quispe

Estudiante: Mogrovejo Ccorimanya Luis Angel (220961)

Cusco — Perú

2026

Resumen ejecutivo

GeoTrack es un sistema integral de geolocalización en tiempo real compuesto por tres piezas que trabajan juntas: una **aplicación móvil Android** (Flutter) que actúa como rastreador, un **backend en la nube de AWS** (Node.js, Socket.IO y PostgreSQL) que recibe, almacena y retransmite las posiciones, y un **centro de monitoreo web** (React y Leaflet) que muestra en vivo todos los dispositivos, su historial y la reproducción animada de sus recorridos.

El teléfono funciona como un *sistema embebido*: un sensor (receptor GNSS), un proceso residente que sobrevive al cierre de la aplicación y al bloqueo de pantalla, almacenamiento local para operar sin conexión, y un canal de comunicación que sincroniza con el servidor cuando hay red. Sobre las lecturas crudas del GPS se implementó un **filtro de precisión** (validación por exactitud y por velocidad física, filtro de Kalman con predicción por velocidad y rumbo, y fijación de posición estando quieto) que eliminó los "saltos" a otros lugares y redujo un 80 % el movimiento falso con el usuario detenido.

1 s intervalo mínimo de captura	**50 785** posiciones registradas en campo	**21** sesiones de rastreo reales
2 servicios AWS (EC2 + RDS)	**3 900** líneas de código propio	**−80 %** deriva falsa estando quieto

Índice

Resumen ejecutivo **1**

1. Introducción **3**
1.1. Problema . . . 3
1.2. Objetivo general . . . 3
1.3. Objetivos específicos . . . 3
1.4. Alcance . . . 3

2. Requerimientos **3**

3. Arquitectura del sistema **4**
3.1. Tecnologías utilizadas . . . 5

4. Infraestructura en la nube (AWS) **5**
4.1. Seguridad de la red . . . 6
4.2. Costo estimado . . . 6

5. Aplicación móvil: el rastreador **6**
5.1. Organización del código . . . 6
5.2. Rastreo persistente en segundo plano . . . 6
5.3. Arquitectura *offline-first* y sincronización . . . 7
5.4. Filtro de precisión de la señal GNSS . . . 8
5.5. Telemetría en el dispositivo . . . 10
5.6. Interfaz móvil . . . 10

6. Backend: API y tiempo real **11**
6.1. Modelo de datos . . . 11
6.2. API REST . . . 12
6.3. Procesamiento de cada lote . . . 12
6.4. Geocercas . . . 12

7. Centro de monitoreo web **13**
7.1. Funcionalidades . . . 13

8. Innovaciones (Fase 5) **14**

9. Despliegue y operación **14**
9.1. Scripts de infraestructura . . . 14
9.2. Problemas encontrados y soluciones . . . 15

10. Resultados y pruebas de campo **15**
10.1. Evaluación del filtro de precisión . . . 15

11. Conclusiones **16**
11.1. Trabajo futuro . . . 16

Referencias **17**

1. Introducción

1.1. Problema

Rastrear la posición de personas o vehículos de forma continua es una necesidad común en logística, seguridad personal, deporte y supervisión de personal en campo. Las soluciones comerciales suelen ser cerradas, costosas o dependientes de una conexión permanente. Además, un teléfono móvil impone restricciones propias de un sistema embebido: batería limitada, un sistema operativo que *mata* procesos en segundo plano para ahorrar energía, conectividad intermitente y un sensor GNSS cuya precisión varía entre 3 y más de 100 metros según el entorno.

1.2. Objetivo general

Desarrollar y desplegar un sistema de geolocalización en tiempo real, de extremo a extremo, compuesto por una aplicación móvil rastreadora, un backend en la nube y una plataforma web de monitoreo, con una interfaz moderna inspirada en Google Maps.

1.3. Objetivos específicos

1. Capturar la ubicación de forma persistente en segundo plano (intervalo configurable de 1 a 10 s), incluso con la aplicación cerrada o el teléfono bloqueado.
2. Garantizar que ningún dato se pierda sin conexión mediante una arquitectura *offline-first*.
3. Calcular telemetría en el dispositivo: velocidad, distancia, tipo de actividad y calorías.
4. Visualizar en vivo múltiples dispositivos, consultar su historial y reproducir sus recorridos.
5. Desplegar la solución en AWS con infraestructura reproducible mediante scripts.
6. Mejorar la calidad de la señal GNSS mediante filtrado y suavizado de las lecturas.

1.4. Alcance

El sistema se organizó en cinco fases: (1) entorno local y despliegue en un dispositivo físico, (2) infraestructura en AWS, (3) aplicación móvil, (4) plataforma web y (5) innovaciones adicionales. La aplicación se probó en un Samsung Galaxy A26 real, en la ciudad del Cusco.

2. Requerimientos

ID	Requerimiento funcional	Estado
RF-01	Mapa a pantalla completa con botón flotante *Iniciar/Detener* rastreo	✅
RF-02	Rastreo persistente en segundo plano (app cerrada o pantalla bloqueada)	✅
RF-03	Almacenamiento local sin conexión y envío de la cola al recuperar la red	✅
RF-04	Historial en el dispositivo: sesiones, ruta dibujada y nombres personalizados	✅
RF-05	Telemetría: velocidad, calorías y clasificación caminata / alta velocidad	✅
RF-06	Panel web en vivo con todos los dispositivos activos y color distintivo	✅
RF-07	Trazado de rutas en tiempo real	✅
RF-08	Consultas históricas por dispositivo, fecha y hora	✅
RF-09	Reproductor de rutas con velocidades x10, x20 y x30 (se añadió x1 y x60)	✅
RF-10	Al menos tres funcionalidades innovadoras (se implementaron seis)	✅

Cuadro 1: Requerimientos funcionales y su cumplimiento.

ID	Requerimiento no funcional
RNF-01	Diseño moderno y minimalista inspirado en Google Maps (tarjetas flotantes, paleta neutra).
RNF-02	Tolerancia a fallos de red: cero pérdida de datos e idempotencia en el servidor.
RNF-03	Tiempo real: las posiciones aparecen en el panel en cuanto llegan al servidor (WebSockets).
RNF-04	Seguridad: autenticación de dispositivos, base de datos privada y cifrada, mínimo privilegio en red.
RNF-05	Despliegue reproducible: toda la infraestructura se crea, actualiza y destruye con scripts.
RNF-06	Eficiencia energética: un solo flujo de GPS y envío por lotes.

Cuadro 2: Requerimientos no funcionales.

3. Arquitectura del sistema

La Figura 1 muestra los tres componentes y cómo se comunican. El teléfono envía las posiciones por HTTP (REST) al servidor en AWS; el servidor las guarda en PostgreSQL y las retransmite al instante a todos los navegadores conectados mediante WebSockets (Socket.IO).

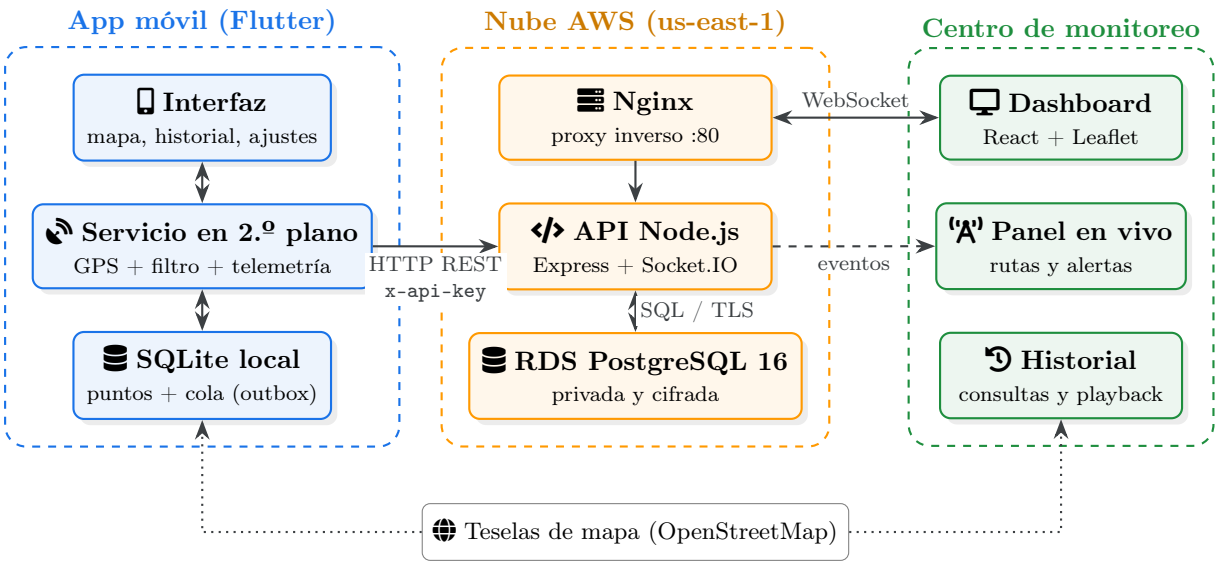


Figura 1: Arquitectura general de GeoTrack.

3.1. Tecnologías utilizadas

Capa	Tecnología	Por qué se eligió
Móvil	Flutter 3 / Dart	Un solo código, compilación nativa (AOT) y alto rendimiento gráfico.
	`flutter_background_service`	Servicio en primer plano de Android en un *isolate* propio.
	`geolocator`	Acceso al proveedor de ubicación fusionado de Android (GNSS + Wi-Fi + celdas).
	`sqflite`	Base de datos SQLite para operar sin conexión.
	`flutter_map`	Mapa vectorial/teselas sin depender de claves de pago.
Backend	Node.js + Express	API REST ligera y asíncrona.
	Socket.IO	WebSockets con reconexión automática para el tiempo real.
	PostgreSQL 16	Integridad, índices por tiempo y restricción `UNIQUE` para idempotencia.
Web	React 18 + Vite	Interfaz reactiva y compilación rápida.
	Leaflet / React-Leaflet	Mapas interactivos con polilíneas, círculos y marcadores.
Nube	AWS EC2 + RDS	Servidor con conexiones persistentes y base de datos gestionada con backups.
	AWS CLI + PowerShell	Infraestructura reproducible por scripts.

Cuadro 3: Pila tecnológica.

4. Infraestructura en la nube (AWS)

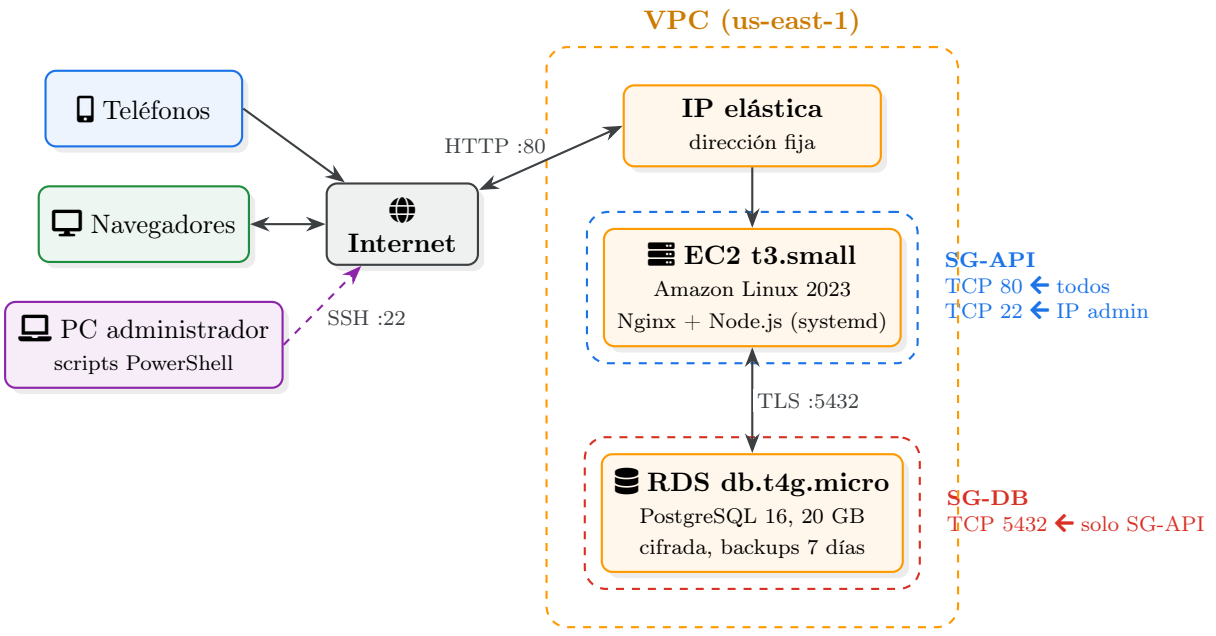


Figura 2: Despliegue en AWS con segmentación de red por Security Groups.

Decisión de diseño: EC2 en lugar de API Gateway + Lambda

Socket.IO necesita **conexiones persistentes** con cada navegador. Un servidor EC2 detrás de Nginx las mantiene de forma natural, con un costo predecible y una arquitectura simple de operar. La base de datos se dejó en RDS para delegar en AWS los backups, los parches y el cifrado.

4.1. Seguridad de la red

- **Mínimo privilegio:** la base de datos no tiene IP pública; solo acepta conexiones desde el Security Group de la API.
- **SSH restringido** a la IP pública del administrador (regla `/32`).
- **Cifrado en reposo** del almacenamiento de RDS y conexión TLS entre la API y la base de datos.
- **Secretos fuera del repositorio:** token del panel, API key de dispositivos y URL de la base de datos se guardan en `infra/.deploy.json`, ignorado por git.

4.2. Costo estimado

Recurso	Tipo	US$/mes
EC2	t3.small (2 vCPU, 2 GB)	≈ 15
RDS PostgreSQL	db.t4g.micro + 20 GB gp2	≈ 13
Almacenamiento, IP elástica y transferencia	—	≈ 2
Total		**$\approx$ 30**

Cuadro 4: Costo mensual aproximado (puede ser menor dentro de la capa gratuita).

5. Aplicación móvil: el rastreador

5.1. Organización del código

La aplicación separa la interfaz del trabajo pesado. El rastreo vive en un *isolate* independiente (un hilo de Dart con memoria propia) dentro de un **servicio en primer plano** de Android, de modo que cerrar la interfaz no detiene la captura.

5.2. Rastreo persistente en segundo plano

Android cierra agresivamente los procesos en segundo plano, y más aún los fabricantes como Samsung. Para evitarlo se combinaron varias técnicas:

- **Servicio en primer plano** con `foregroundServiceType="location"` y notificación permanente que muestra velocidad, distancia, calorías y puntos en cola.
- `stopWithTask=false`: el servicio sigue vivo aunque se deslice la app fuera de recientes.
- `autoStartOnBoot`: si el teléfono se reinicia con una sesión activa, el rastreo se reanuda.
- Permiso de ubicación *"Permitir todo el tiempo"* y exclusión de la optimización de batería.
- **Watchdog** cada 30 s: si el flujo de GPS deja de entregar posiciones (algunos fabricantes lo pausan), fuerza una lectura y reinicia el flujo.

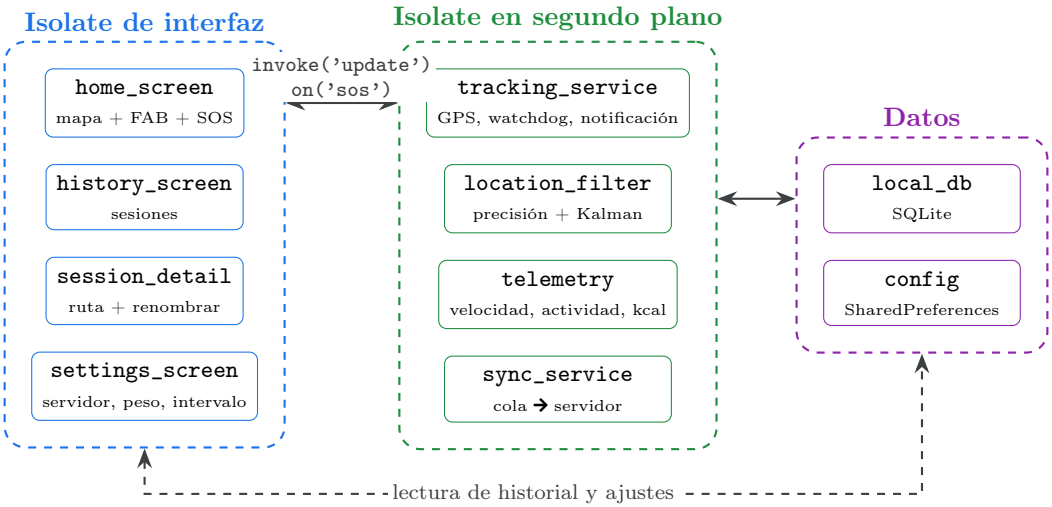


Figura 3: Módulos de la app y comunicación entre isolates.

5.3. Arquitectura *offline-first* y sincronización

Cada posición se guarda **primero en SQLite** y solo después se intenta enviar. Las operaciones que no son puntos (iniciar/detener sesión, renombrar, SOS) van a una tabla `outbox` y se envían en orden.

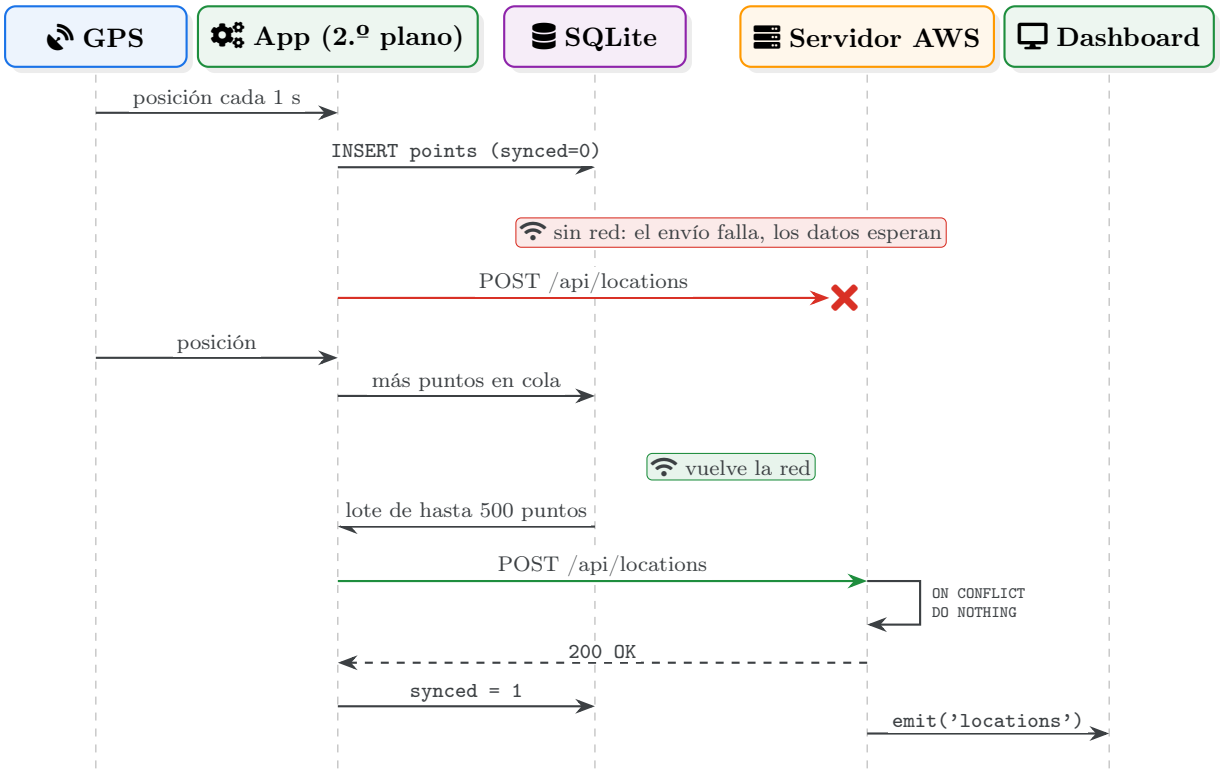


Figura 4: Diagrama de secuencia: corte de red y recuperación sin pérdida de datos.

Idempotencia

La tabla `locations` tiene la restricción `UNIQUE(device_id, ts)` y la inserción usa `ON CONFLICT DO NOTHING`. Si un lote llega dos veces (por ejemplo, el servidor lo guardó pero la respuesta se perdió por un corte de red), los puntos repetidos se ignoran: los reintentos nunca duplican datos.

Listing 1: Inserción idempotente de un lote de posiciones.

```
INSERT INTO locations (device_id, session_id, ts, lat, lng, speed, accuracy,
    ...)
VALUES ($1, $2, $3, ...), ($12, $13, $14, ...), ...
ON CONFLICT (device_id, ts) DO NOTHING
RETURNING *;
```

5.4. Filtro de precisión de la señal GNSS

Durante las pruebas de campo se observó que la ruta "saltaba" a otros lugares. El análisis de **24 351 posiciones reales** mostró que todos esos saltos coincidían con lecturas que el propio teléfono reportaba con un margen de error de 74 a 800 m: ubicaciones aproximadas por Wi-Fi o antenas celulares, no por satélite. La Figura 5 muestra la distribución.

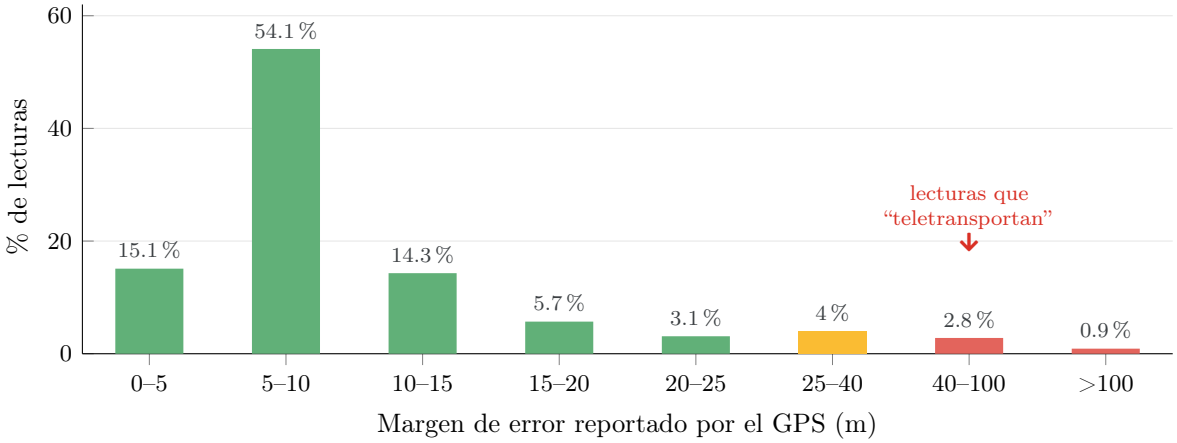


Figura 5: Distribución de la precisión en 24 351 lecturas reales. En verde: aceptadas siempre; amarillo: aceptadas solo tras 20 s sin una lectura mejor; rojo: descartadas.

Para resolverlo se diseñó un filtro de cuatro etapas que procesa cada lectura antes de guardarla, enviarla o dibujarla (Figura 6).

Filtro de Kalman con predicción. Para cada coordenada se mantiene una estimación $\hat{x}$ y su varianza P (en m^2). Con un intervalo Δt, una velocidad v y un rumbo θ reportados por el GPS (efecto Doppler), y una medición z de precisión σ:

$$\hat{x}^- = \hat{x} + v\,\Delta t\,(\cos\theta,\ \sin\theta) \qquad \text{predicción por velocidad y rumbo} \tag{1}$$

$$P^- = P + \Delta t\, q^2, \qquad q = 2 + 0.5\,v \qquad \text{incertidumbre del movimiento} \tag{2}$$

$$K = \frac{P^-}{P^- + \sigma^2} \qquad \text{ganancia: cuánto confiar en la medición} \tag{3}$$

$$\hat{x} = \hat{x}^- + K\,(z - \hat{x}^-), \qquad P = (1 - K)\,P^- \qquad \text{corrección} \tag{4}$$

Predecir con la velocidad evita que el punto "se quede atrás" en un vehículo, y hacer que q crezca con la velocidad permite seguir curvas y frenadas.

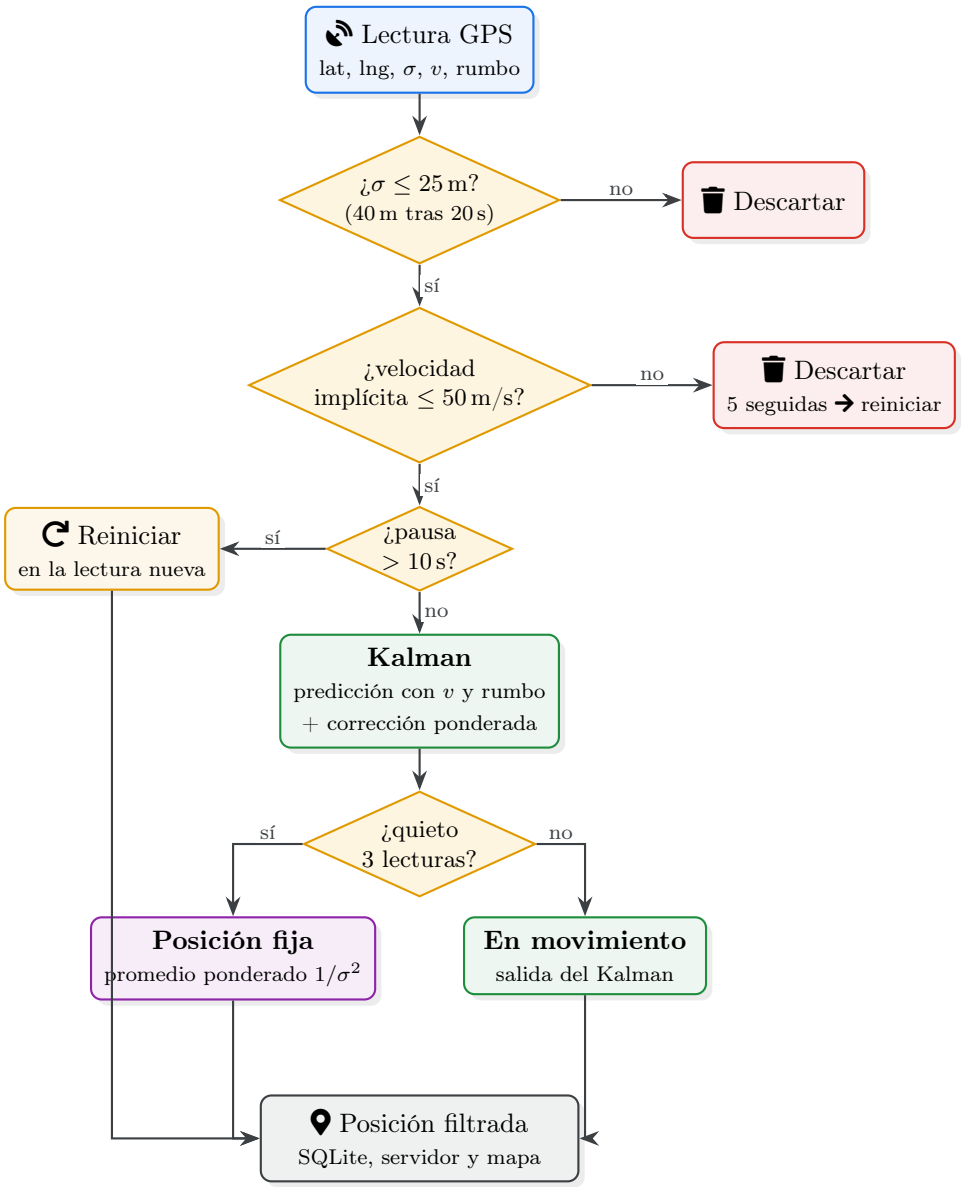


Figura 6: Etapas del filtro de precisión (`location_filter.dart`).

Posición fija estando quieto. Cuando la velocidad es menor a 0.5 m/s durante tres lecturas, la posición se fija en un promedio ponderado con olvido gradual ($\lambda = 0.98$, memoria de unos 50 s), que da más peso a las lecturas precisas:

$$\bar{x} = \frac{\sum_i \lambda^{n-i}\, w_i\, z_i}{\sum_i \lambda^{n-i}\, w_i}, \qquad w_i = \frac{1}{\sigma_i^2} \tag{5}$$

El modo se abandona cuando la velocidad supera 1 m/s o dos lecturas seguidas caen lejos del punto fijo.

Listing 2: Núcleo del filtro de Kalman (Dart).

```dart
// Predicción: avanzar según la velocidad y rumbo que reporta el GPS (Doppler).
if (speed != null && speed > 0.5 && heading != null && heading >= 0) {
  final step = speed * dt;
  final h = heading * pi / 180;
  _lat = _lat! + (step * cos(h)) / 111320.0;
  _lng = _lng! + (step * sin(h)) / (111320.0 * cos(_lat! * pi / 180));
}
final q = processNoise + 0.5 * max(speed ?? 0, 0);
_variance += dt * q * q;

// Corrección con la medición, ponderada por su precisión.
final k = _variance / (_variance + acc * acc);
_lat = _lat! + k * (lat - _lat!);
_lng = _lng! + k * (lng - _lng!);
_variance = (1 - k) * _variance;
```

5.5. Telemetría en el dispositivo

La velocidad se toma del GPS (efecto Doppler, más estable que distancia/tiempo) y se suaviza con la mediana de las tres últimas lecturas. Con ella se clasifica la actividad y se estiman las calorías:

$$\text{kcal} = \text{MET}(\text{actividad}, v) \times \text{peso (kg)} \times \text{tiempo (h)} \tag{6}$$

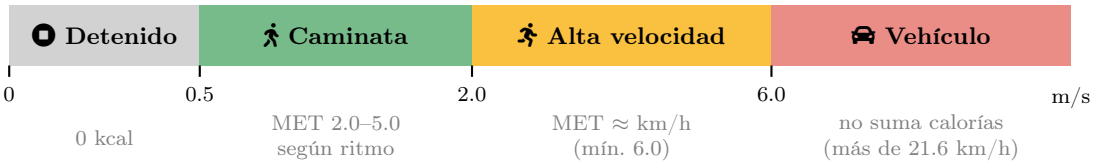
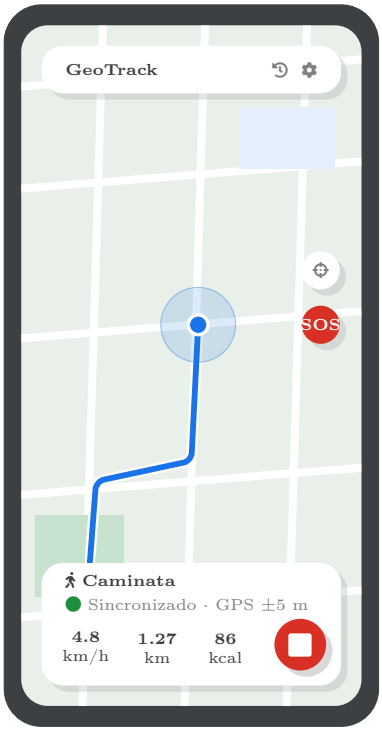


Figura 7: Clasificación de la actividad por velocidad y MET asociado (Compendium of Physical Activities).

Para no inflar la distancia, los desplazamientos solo se suman si la lectura es precisa ($\sigma \leq 35$ m) y hay movimiento real (velocidad > 0.5 m/s o desplazamiento mayor que el margen de error).

5.6. Interfaz móvil

La interfaz sigue la estética de Google Maps: el mapa ocupa toda la pantalla y los controles flotan encima en tarjetas blancas con sombra suave.



- **Ruta en vivo** dibujada con borde blanco, como en Google Maps.
- **Punto azul con círculo de precisión**: el radio es el margen de error del GPS en ese momento.
- **Línea de estado**: rastreo, sincronización, puntos en cola y precisión (“GPS ±5 m”).
- **Tarjeta de telemetría**: velocidad, distancia, calorías y actividad.
- **Botón SOS**: se mantiene presionado 2 s para evitar activaciones accidentales.
- **Botón flotante** Iniciar/Detener.

Figura 8: Pantalla principal de la aplicación móvil.

6. Backend: API y tiempo real

6.1. Modelo de datos

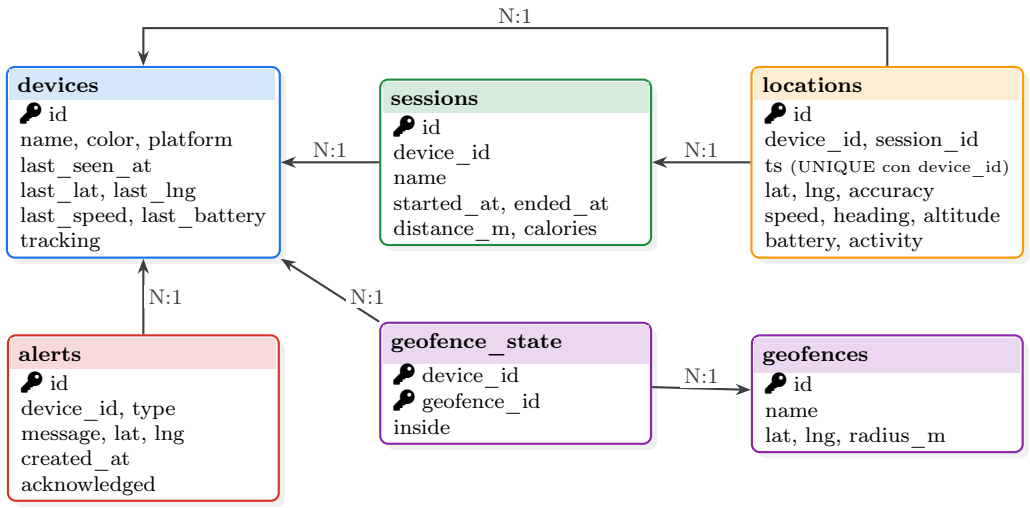


Figura 9: Modelo entidad-relación en PostgreSQL.

6.2. API REST

Método	Ruta	Descripción
Dispositivos (cabecera `x-api-key`)		
POST	`/api/devices/register`	Registra el teléfono y le asigna un color único.
POST	`/api/sessions/start`	Inicia una sesión de rastreo.
POST	`/api/sessions/:id/stop`	Cierra la sesión con distancia y calorías finales.
PATCH	`/api/sessions/:id`	Renombra una sesión.
POST	`/api/locations`	Recibe un lote de posiciones (idempotente).
POST	`/api/alerts/sos`	Emite una alerta SOS con la última posición.
Dashboard		
GET	`/api/devices`, `/api/live`	Estado de los dispositivos y rutas de las sesiones activas.
GET	`/api/history`	Posiciones de un dispositivo en un rango de fechas.
GET	`/api/history/stats`	Distancia, duración, velocidad media y máxima.
GET	`/api/history/export`	Descarga del recorrido en GPX o CSV.
GET/POST/DELETE	`/api/geofences`	Administración de geocercas.
GET/POST	`/api/alerts`, `/:id/ack`	Listado y confirmación de alertas.

Cuadro 5: Principales rutas de la API.

6.3. Procesamiento de cada lote

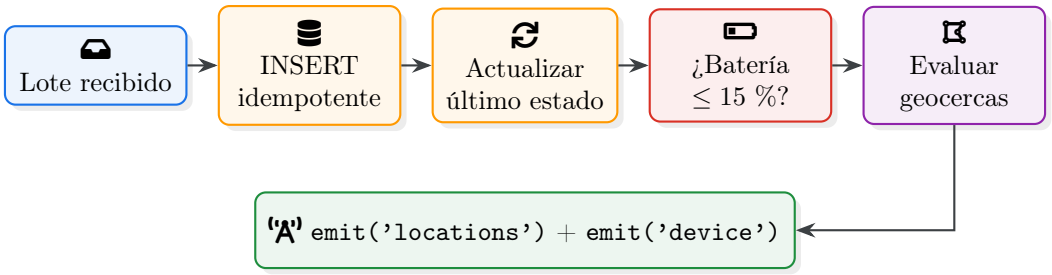


Figura 10: Flujo de `POST /api/locations` en el servidor.

Además, un **vigilante** se ejecuta cada 30 s y genera una alerta *"sin señal"* si un dispositivo que está rastreando lleva más de 2 minutos sin enviar datos.

6.4. Geocercas

Una geocerca es un círculo (centro y radio). Para cada posición se calcula la distancia al centro con la fórmula de Haversine, y se compara con el último estado conocido (dentro/fuera) guardado en `geofence_state`; solo un *cambio* de estado genera alerta:

$$d = 2R \arcsin\sqrt{\sin^2\frac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2\frac{\Delta\lambda}{2}}, \qquad R = 6\,371\,000 \text{ m} \tag{7}$$

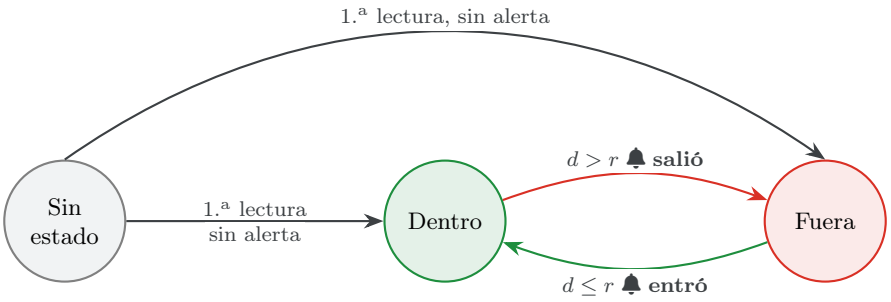


Figura 11: Máquina de estados de una geocerca por dispositivo.

7. Centro de monitoreo web

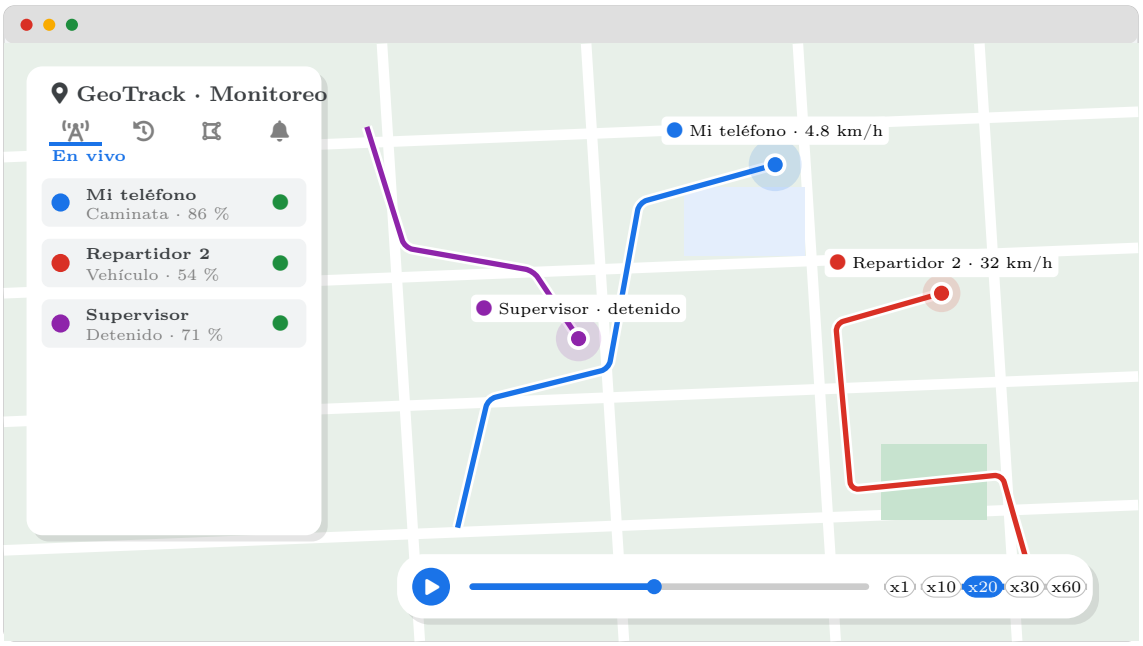


Figura 12: Centro de monitoreo: panel en vivo con varios dispositivos y reproductor de rutas.

7.1. Funcionalidades

- **Panel en vivo:** todos los dispositivos que rastrean, cada uno con su color, etiqueta con nombre y velocidad, y círculo de precisión alrededor de la última posición.
- **Trazado en tiempo real:** cada evento `locations` de Socket.IO extiende la polilínea del dispositivo sin recargar la página.
- **Consultas históricas:** filtros por dispositivo, fecha y hora, con atajos (1 h, Hoy, 24 h, 7 días) y estadísticas del período.
- **Reproductor (playback):** Play/Pausa, barra de progreso arrastrable y velocidades x1, x10, x20, x30 y x60; los huecos sin señal se saltan para no esperar en vano.
- **Exportación** del recorrido en GPX (compatible con Google Earth y Strava) o CSV.

8. Innovaciones (Fase 5)

	Funcionalidad	Valor que aporta
	Geocercas con alertas	Avisa cuando un dispositivo entra o sale de una zona (almacén, colegio, obra). Se dibujan con un clic y un radio.
	Botón SOS	Alerta roja inmediata en el panel con la posición exacta; se mantiene 2 s para evitar falsas alarmas.
	Alertas automáticas	Batería baja ($\leq$ 15 %) y dispositivo *sin señal* (> 2 min sin datos).
	Exportación GPX/CSV y estadísticas	Distancia, duración, velocidad media y máxima; integración con otras herramientas.
	Autenticación	API key para dispositivos y token para el panel.
	Filtro de precisión GNSS	Elimina saltos, suaviza la ruta, fija la posición estando quieto y muestra el margen de error.

Cuadro 6: Funcionalidades adicionales implementadas.

9. Despliegue y operación

9.1. Scripts de infraestructura

Script	Qué hace
`deploy-aws.ps1`	Crea llave SSH, Security Groups, RDS, EC2 con Nginx y Node.js, IP elástica, y despliega la aplicación (unos 15 min).
`update-app.ps1`	Compila el dashboard, empaqueta el backend, lo sube por SSH, reinicia el servicio y verifica `/api/health`.
`build-apk-cloud.ps1`	Alternativa: compila el APK en una EC2 Linux temporal que se autodestruye.
`destroy-aws.ps1`	Elimina todos los recursos para dejar de pagar.

Cuadro 7: Automatización del ciclo de vida de la infraestructura.

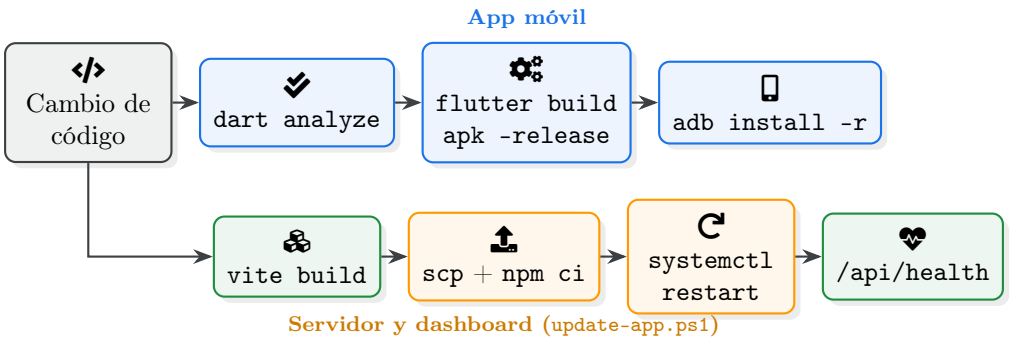


Figura 13: Flujo de entrega de una nueva versión.

Firma del APK

El APK de producción se firma siempre con la misma llave (`geotrack-release.jks`). Así cada actualización se instala encima de la anterior sin borrar el historial guardado en el teléfono.

9.2. Problemas encontrados y soluciones

Problema	Causa	Solución
La ruta "salta" a otros lugares	Lecturas por Wi-Fi/antenas con error de 74–800 m tratadas como exactas.	Filtro de precisión de cuatro etapas (Sección 5.4).
El punto se mueve estando quieto	Ruido normal del GNSS de 5–20 m.	Kalman + promedio ponderado en modo quieto.
Aviso falso "Sin conexión"	La pantalla solo se refrescaba al llegar una posición nueva.	Refresco tras cada envío y estado inicial "desconocido".
No compila en Windows	La tilde de la carpeta ("posición") rompe el compilador AOT de Dart.	Unidad virtual sin tildes (`subst G:`) y caché incremental de Kotlin desactivada.
Scripts colgados en SSH	Cambió la IP pública del administrador.	Nueva regla `/32` en el Security Group.
Android pausa el GPS	Optimización de batería del fabricante.	Servicio en primer plano + watchdog cada 30 s.

Cuadro 8: Incidencias técnicas y su resolución.

10. Resultados y pruebas de campo

El sistema se probó durante varios días en la ciudad del Cusco con un Samsung Galaxy A26 (receptor GNSS Samsung S.LSI), caminando, en vehículo y dentro de edificios.

Indicador	Valor
Dispositivos registrados	3
Sesiones de rastreo	21
Posiciones almacenadas en el servidor	50 785
Distancia acumulada en sesiones	97.8 km
Velocidad máxima registrada (vehículo)	71.4 km/h
Lecturas con precisión ≤ 10 m	69.2 %
Intervalo mediano entre puntos (configurado a 1 s)	1.0 s

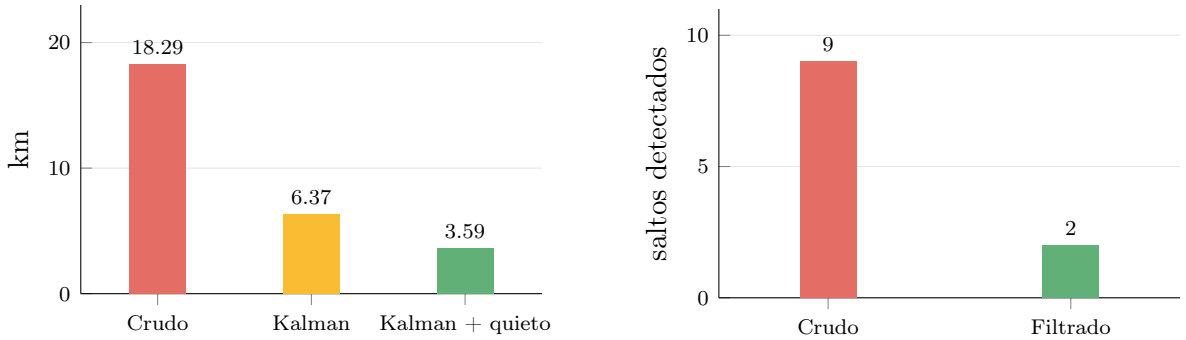
Cuadro 9: Resumen de las pruebas de campo.

10.1. Evaluación del filtro de precisión

Las 24 351 lecturas crudas del 8 y 9 de octubre se volvieron a procesar con el filtro para medir su efecto de forma objetiva:

Métrica	Sin filtro	Con filtro
Saltos a otro lugar	9	2*
Movimiento falso estando quieto	18.29 km	3.59 km (−80 %)
Desfase respecto al GPS en vehículo (percentil 95)	—	5.9 m
Lecturas descartadas por imprecisas	0 %	6.8 %

Cuadro 10: Métricas del filtro. *Los dos restantes corresponden a vueltas en U reales después de que el GPS se pausara.



(a) Movimiento falso acumulado estando quieto.

(b) Saltos a otro lugar y regreso.

Figura 14: Efecto del filtro sobre datos reales.

⚠ Límite físico

Un receptor GNSS de teléfono de una sola frecuencia tiene un límite de unos 3–5 m al aire libre y peor entre edificios o bajo techo, por la reflexión de la señal (*multipath*). Ningún software lo elimina por completo; para precisión centimétrica se requieren equipos con corrección RTK. Por eso la app muestra el **círculo de precisión**: comunica honestamente cuánto confiar en cada lectura.

11. Conclusiones

1. Se construyó y desplegó un sistema de rastreo completo y funcional de extremo a extremo: app Android, backend en AWS y centro de monitoreo web, que cumple los diez requerimientos funcionales y supera el mínimo de innovaciones solicitado.
2. El enfoque *offline-first* con idempotencia en el servidor permite que los cortes de red no pierdan ni dupliquen datos: todo se guarda primero en el teléfono y se reenvía al volver la señal, algo esencial en un sistema embebido móvil con conectividad intermitente.
3. El servicio en primer plano con watchdog, independiente de la interfaz, permitió un rastreo continuo de un punto por segundo (intervalo mediano medido en campo: 1.0 s).
4. El análisis de datos reales permitió diagnosticar el origen de los errores de posición y diseñar un filtro (validación, Kalman y promedio estando quieto) que eliminó los saltos y redujo un 80 % la deriva estando quieto.
5. La automatización con scripts hizo la infraestructura reproducible y redujo cada actualización del servidor a un solo comando.

11.1. Trabajo futuro

- **HTTPS** con un dominio propio y certificado de Let's Encrypt (`certbot`).
- **Ajuste a calles** (*map matching*) para pegar la ruta del vehículo a la vía.
- Kalman de estado completo (posición y velocidad) y fusión con acelerómetro y giroscopio.
- Notificaciones *push* de las alertas al teléfono del supervisor.
- Cuentas de usuario con roles y organizaciones (multiinquilino) y versión para iOS.
- Frecuencia de captura adaptativa según el movimiento, para ahorrar batería.

Referencias

[1] Flutter. *Documentación oficial*. https://docs.flutter.dev

[2] Android Developers. *Foreground services* y *Background location limits*. https://developer.android.com/develop/background-work/services/fgs

[3] Amazon Web Services. *Amazon EC2* y *Amazon RDS for PostgreSQL User Guide*. https://docs.aws.amazon.com

[4] Socket.IO. *Documentation*. https://socket.io/docs/v4/

[5] Leaflet. *An open-source JavaScript library for interactive maps*. https://leafletjs.com

[6] Ainsworth, B. E. et al. (2011). *Compendium of Physical Activities*. Medicine & Science in Sports & Exercise, 43(8), 1575–1581.

[7] Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1), 35–45.

[8] Sinnott, R. W. (1984). Virtues of the Haversine. *Sky and Telescope*, 68(2), 159.